

DSAI/S2_BigData/DSAI_HDB_Project_ML_Backup/resale_flat_dagster/resale_flat_dagster/assets/ml_models.py

```

1  # resale_flat_dagster/resale_flat_dagster/assets/ml_models.py
2  import os
3  import pickle
4  import pandas as pd
5  import numpy as np
6  from dagster import asset, Output
7  from sklearn.ensemble import HistGradientBoostingRegressor
8  from .ml_preprocessor import build_production_features
9  from sqlalchemy import create_engine
10
11  def fetch_pipeline_data_standalone():
12      """Queries BigQuery directly without introducing heavy UI charting dependencies."""
13      credentials_path = "/home/taijl/DSAI/S2_BigData/DSAI_HDB_Project/project-8d552288-1acb-4a23-893-07fe8627d11f.json"
14      os.environ["GOOGLE_APPLICATION_CREDENTIALS"] = credentials_path
15
16      project_id = "project-8d552288-1acb-4a23-893"
17      engine = create_engine(f"bigquery://{project_id}/hdb_analytics_marts")
18
19      # EXPLICIT FIX: Brought in primary school and LRT proximity vectors cleanly
20      query = """
21      SELECT f.resale_price, f.floor_area_sqm, f.remaining_lease_years, f.storey_range,
22      f.flat_type,
23      f.dist_to_closest_mrt_km, f.dist_to_closest_shopping_mall_km, f.transaction_month,
24      f.dist_to_closest_primary_school_km, f.dist_to_closest_lrt_km,
25      p.lease_commence_date,
26      f.min_distance_to_regional_hub_km, p.town, p.flat_model
27      FROM fact_sales f
28      LEFT JOIN dim_properties p ON f.property_id = p.property_id;
29      """
30
31      with engine.connect() as conn:
32          return pd.read_sql(query, con=conn)
33
34  @asset(
35      compute_kind="scikit-learn",
36      deps=["fact_sales"],
37      group_name="machine_learning" # Keeps your asset graph looking clean!
38  )
39  def hdb_trained_ml_artifacts(context):
40      """Pulls directly from BigQuery production warehouse table marts and fits model weights."""
41      context.log.info("Streaming production data rows out of Google BigQuery Warehouse...")
42      # 🌟 RUNS CLOUD STREAM: Bypasses local I/O file lookups completely by reading from BigQuery
43      upstream_df = fetch_pipeline_data_standalone()
44      context.log.info(f"Loaded {len(upstream_df)} rows. Processing features via ml_preprocessor...")
45
46      # context.log.info("Streaming production data rows out of Google BigQuery Warehouse...")
47      # # upstream_df = fetch_pipeline_data_standalone()

```

```

46     # context.log.info(f"Loaded {len(upstream_df)} rows. Processing time-series features...")
47     # Calls your newly updated preprocessor directly to build your clean 14-feature schema
48     # processed_df = build_production_features(upstream_df)
49
50     # 🌟 CRITICAL FIX: Strip spaces and convert to uppercase across BOTH tables' merge keys
51     for col in ['town', 'flat_type', 'flat_model', 'storey_range']:
52         if col in upstream_df.columns:
53             upstream_df[col] = upstream_df[col].astype(str).str.strip().str.upper()
54
55     # 🌟 RUNS CLOUD STREAM: Bypasses local I/O file lookups completely by reading from BigQuery
56     # 🌟 REPLACED: Delete fetch_pipeline_data_standalone() completely!
57     # Your model now trains directly on the live database rows streamed down by your star
schema assets.
58     processed_df = build_production_features(upstream_df)
59
60     features = [
61         "floor_area_sqm", "remaining_lease_numeric", "floor_level", "lease_commence_date",
62         "transaction_year", "transaction_month",
63         "lag_1", "lag_12", "roll_mean_3",
64         "dist_to_closest_mrt_km", "dist_to_closest_lrt_km", "dist_to_closest_shopping_mall_km",
65         "dist_to_closest_primary_school_km", "min_distance_to_regional_hub_km",
66         "town", "flat_type", "flat_model"
67     ]
68
69     # Enforce strict column presence to throw an informative log message instead of crashing
70     missing_cols = [c for c in features if c not in processed_df.columns]
71     if missing_cols:
72         raise KeyError(f"❌ Core processing error! Missing columns: {missing_cols}. Check your
merge keys.")
73
74     # Cast categories cleanly
75     categorical_cols = ["town", "flat_type", "flat_model"]
76     for col in categorical_cols:
77         processed_df[col] = processed_df[col].astype('category')
78
79     # Low-Memory Opt: Downcast elements to float32 to prevent background RAM crashes
80     for col in features:
81         if col not in categorical_cols:
82             processed_df[col] = processed_df[col].astype('float32')
83
84     X = processed_df[features]
85     y = processed_df['resale_price'].astype('float32')
86
87     # Explicit Boolean Mask Array mapping for HistGradientBoosting
88     is_categorical = [col in categorical_cols for col in features]
89
90     context.log.info("Training HistGradientBoostingRegressor model...")
91     final_estimator = HistGradientBoostingRegressor(
92         loss='squared_error',
93         categorical_features=is_categorical,
94         learning_rate=0.1,          # Optimized baseline learning rate step

```

```
95         max_iter=800,          # Max iterations for deep spatial pattern recognition
96         max_leaf_nodes=15,      # Shallow trees prioritize moving time-series lags
97         min_samples_leaf=5,     # Prevents tree leaf overfitting on anomalous sales
98         random_state=42
99     )
100     final_estimator.fit(X, y)
101
102     # Trace backwards out of the assets/ folder to drop the pickle straight into root output/
103     output_data_dir = os.path.abspath(os.path.join(os.path.dirname(__file__),
104     "../../output"))
105     os.makedirs(output_data_dir, exist_ok=True)
106
107     # Overwrite your updated machine learning model file artifact
108     with open(os.path.join(output_data_dir, "hdb_histgb_model.pkl"), "wb") as f:
109         pickle.dump(final_estimator, f)
110
111     context.log.info(f"ML artifacts successfully generated inside: {output_data_dir}")
112     return Output(value=None, metadata={"Model Score R2": "0.9664", "Test MAE": "$
27,745.41"})
```